格拉斯哥大学计算机科学学院

课程作业评估

课程名称	编程 COMPSCI4039			
课程作业编号	1/1			
截止日期	时间:	16:30	日期:	20/11/2026
对最终课程成绩的贡献比例	8%			
个人或团体	独奏	☒	团队	
预计工时	平均40小时			
提交说明	提交运行游戏所需的所有代码。			
请注意：本课程作业不可重新评估				

课程作业提交评估规则

需正式评估的课程作业提交截止日期将公布在课程资料中，逾期提交的作业将按以下规定受到处罚。

对于在公布截止日期后提交的课程作业，其主要成绩和次要等级将按如下方式计算：

(i) 对于在截止日期后不超过五个工作日内提交的作品
   a. 作业将按常规方式进行评估；
   b. 确定的初级等级和中级等级将根据作业提交延迟的工作日（或部分工作日）数量，每延迟一天（或部分工作日）减少两个中级等级。

（ii）超过截止日期五个工作日后提交的作业将被评定为H等级。

如能证明有正当理由导致作业迟交，则不会受到处罚。您应通过MyCampus提交支持正当理由的相关文件。

未遵守投稿说明的处罚为：降低两个等级

COMPSCI 4039 – 编程（信息技术）

评估练习——生态系统拯救者

1. 引言——你为什么要这么做？这属于什么范畴？

本练习旨在让你展示你在理解与开发程序方面日益提升的能力，运用公认的编程概念和模式，并通过Python编程语言进行表达。

~~鼓励！许多学生会优先选择做这个练习，而不是我们接下来几周布置的其他任务。不要这样做！~~我们提供的所有练习，无论是非考核的实验作业还是本次考核练习，都是按照循序渐进的顺序设计的，旨在帮助你逐步提升能力。实验作业比这个练习更简单，先完成它来锻炼你的技能，然后再着手做这个练习。下面会有关于如何安排你工作的说明。

~~整个练习旨在围绕以下一系列重要概念展开：~~

a. 核心编程概念，如表达式、变量的创建与更新、基本控制流程、文件操作、函数的创建与使用以及库的使用。
b. 文本输入与输出。
c. 数据和活动的面向对象建模，包括类、属性、方法和继承的使用
d. 事件驱动的图形用户交互，包括使用函数作为事件处理程序
e. 小型算法，用 Python 表示，无论是你自己创建的，还是由他人创建后调整以适应你的需求，用于实现你选择的子目标
f. 整个应用程序的模型-视图-控制器（MVC）结构

关联评估。就整体课程作业而言，本次练习与第二项活动相关联，具体方式如下：你需提交此处描述的练习，由我们于第9周结束时进行评分。在第10周的实验课上，我们将开展第二项活动——一次实验测试，你将回答有关代码的问题并进行少量修改，通过在Moodle测验中输入答案以及复制粘贴新代码或更新后的代码来提交。我们将默认你已完成第一阶段。

因此，深入理解你提交的代码以及它与任务之间的关联非常重要。

在继续之前，最后再强调一点：我们始终需要通过一个具体情境下的特定任务（即一种具体的活动）来展示上述所列的各项技能。虽然为所有学生设置统一的练习有许多优势，但我们理解，你们中的一些人可能会觉得某些情境或任务比其他更感兴趣或更具吸引力。但请记住，无论你是否对主题感兴趣，你现在关注的重点是全面展示你在通用知识、技能和理解方面所应具备的高水平能力。这才是这项练习最重要的成果。

好的，那么——我们来讨论一下这个背景……

2. 背景与任务说明

概述

任务是使用 Python 开发一款涉及修复受损生态系统的游戏。

游戏在一个8×8的棋盘上进行。所有描述的游戏元素均占据一个完整的方格。棋盘上包含健康的和受污染的森林区域，设有可提供生态修复材料的环境站点，以及在棋盘上移动并造成污染的污染者角色。

玩家从棋盘的左上角开始——记作 (0,0)。玩家的目标如下：

- 从环境修复站收集生态修复材料（走到修复站的广场上即可获得修复材料）

将修复材料带到病态的森林区域进行修复（携带修复材料进入病态区域即可完成修复，同时消耗掉材料）。

玩家的旅程变得复杂，具体如下：

-

固定的地形特征——山脉和湖泊，玩家无法穿过。

污染者是非玩家角色（NPC），会自动在棋盘上移动——每当它们移动到健康的森林区域时，该区域就会变病。

如果玩家在没有修复材料的情况下进入污染森林，或者污染者移动到玩家所在位置，则游戏结束。

更详细地说

游戏在8×8的棋盘上进行，建议将(0,0)设为棋盘的左上角。
初始布局包含以下方格：

4座山和4个湖泊

5片健康森林和3片病态森林

- 3 个维修站

- 3个污染者和一个玩家角色。

玩家从坐标 (0,0)（左上角）开始。
玩家每次只能携带一袋维修材料

他们通过走到维修站来收集这些物品。

如果他们已经携带了未使用的维修材料，则不会发生任何情况。

当玩家携带修复材料并走到生病的森林上时，会修复该森林。

维修材料用完后，玩家因完成维修而获得一分。

走进健康的森林并不会消耗其资源

如果玩家修复了所有生病的森林方块，将获得2点奖励。
- 如果游戏立即结束

- 一个NPC与玩家落在同一格上，或反之。

一名玩家移动到一个没有修复材料的病态森林格子上。

玩家最初有20次机会，尽可能修复更多的病态方块。

3. 主要任务

第一阶段：文本输入与输出处理（75%）

A部分——电路板设置（15分）

以下是关于如何使用一组提供棋盘模型的类来设置棋盘的指导。（如您稍后将看到的，这是MVC模式中的一个重要部分。）

创建类来表示棋盘上所有类型的方格。你可能需要：

- 一个固定类——其对象可以表示你想要的任何固定特征，例如所描述的山脉和湖泊。
- 一个维修站类
- 森林类别——可能是病态的或健康的
- 一个NPC类
- 玩家类

所有这些类都将有一个棋盘位置，以及一个用于打印棋盘状态的字母/代码（稍后当界面图形化时，我们还可以添加图像）。你能看出这里使用继承的好处吗？比如创建一个基类BoardItem，然后上述所有类都继承自它?

创建一个 Board 类来表示一个 8×8 的游戏棋盘。设置棋盘的一种方法是让构造函数接收一个文件名作为参数。该文件包含用于填充棋盘的数据。文件格式为 8 行，每行包含 8 个以逗号分隔的值，使用以下字符来表示不同类型的棋盘格子：

M – 山地
- L – 湖
- R – 修理站
- SF – 病态森林
HF – 健康森林
- 一个短横线，-，用来表示空格

此格式的一个示例文件是：

```
-,-,-,-,HF,,-,HF
-,R,-,-,-,-,-,-
-,-,L,M,M,-,SF,-
-,HF,M,SF,-,-,-,-
-,M,-,L,-,-,R,-
-,-,-,L,L,-,-,-
-,R,-,-,-,-,HF,-
-,-,SF,HF,-,-,-,-
```

在构造函数内部，你可以逐行读取这些数据，使用 .split() 方法以逗号为分隔符拆分每一行，然后利用生成的列表创建所需的对象，例如用于表示棋盘的嵌套列表。你可以使用 if...elif... elif... 结构来测试 split 产生的列表中的各个元素，从而决定创建哪种类型的对象。

文件表示中不包含任何NPC或玩家。你可以使用随机库生成两个随机数，作为NPC的(x， y)位置。生成的位置可能为

直接在固定物品上，比如一座山——因此你需要设置代码，使其能够生成几个位置，直到出现一个空闲的位置。可以查阅 random.randint() 来生成随机数。

在网格的左上角创建一个玩家对象。

B部分——展示棋盘（5分）

在你的 Board 类中添加一个 printBoard 方法，用于将棋盘状态打印到屏幕上，使用类似上述文件格式中的符号（这些符号可以存储在你的 BoardItem 对象中）。在创建复杂的图形界面之前，你可以在每轮操作后使用此方法来显示棋盘。

这个简单的文本显示是模型-视图-控制器模式中视图部分的第一个版本。

你现在已处于一个很好的位置，可以测试到目前为止的一切是否正常运行。如果你已经完成了上述所有步骤——从文件创建游戏板、添加NPC和玩家，然后打印出游戏板以确认一切正常——这是一项重大的成就，干得好！

C部分——玩家移动（10分）

现在你将开始开发MVC中的控制器部分。这一部分负责处理用户输入，并根据输入决定采取何种操作——依据游戏逻辑更新模型，或刷新视图。

首先，我们只使用文本输入。

在新的 controller.py 文件中，导入你其他文件中的类。

玩家通过输入单个字符（u、d(下)、l(左)、r）来指定方向，并输入一个最多为3的数字，表示移动的格数。请设置如下形式的while循环：

```
获取用户输入
当用户输入不是“q”（用于退出）时：
        如果用户输入等于“u”：
                处理一个上架请求
        elifd":
                处理下架请求
        以及对 l 和 r 依此类推


        距离 = input("... 以及要行驶的距离？ > ")
        现在处理方向和距离
        …

        # 最后——务必获取用户的下一个输入 获取用户
        输入
```

移动以方格为单位进行，遇到棋盘边缘或固定物体时停止。

让移动功能良好运行很有趣。你不想把移动代码放在控制器里。控制器应该负责通知模型来改变玩家的位置。因此，Player 类实际上应该有一个 move 方法。

看看你能不能把这设置好。你可以简化上面的计划，直接将移动指令（上、下、左、右）发送给移动方法，然后由该方法来处理具体操作！

当简单的移动功能实现后，考虑如何处理山脉和水域等无法通行的障碍物，并加以实现。

D部分——污染者NPC移动（10分）

每个NPC每次随机移动1到3格，方向随机，每次移动一格。

第五部分——修复站与森林修复（10分）

为玩家添加修复材料属性，并确保当玩家移动到修复站时该属性能正确更新，同时当玩家手持材料移动到受损的森林时，能够对森林进行适当的修复。

F部分——游戏结束：经过20轮以及与NPC或生病的森林互动后（10分）

如果达到回合限制，则应报告当前得分。另外两种情况将导致立即游戏结束——此时应报告截至该点的得分。

G部分——智能污染者NPC移动（5分）

不要采用随机移动，而是创建一个具有适当继承关系的智能污染者类，使其更直接地朝向健康的森林或玩家移动。设置游戏时，包含一个这样的污染者和两个原本随机移动的NPC。

H部分——面向对象、程序结构与函数（10分）

使用合适的类、继承、函数、有意义的变量名、内部注释以及适当的模块化分解。

第二阶段：使用Tkinter图形用户界面（25%）

此阶段涉及升级控制器，以使用交互式图形用户界面。

第一部分——使用棋子图像绘制棋盘（10分）

第一部分将引导你使用棋子图片来绘制游戏棋盘。我们会提供一些可供使用的图片，但你也可以用自己的图片来定制专属的游戏。

在此阶段，你仍然可以使用文本输入来接收用户的移动指令。错误信息可以通过弹出窗口显示在屏幕上，或以你设计的窗口形式呈现，例如用于显示回合计数。

J部分——使用鼠标或键盘的图形用户界面输入来选择玩家移动（15分）

此阶段最后加入了基于鼠标的交互功能，通过检测Tkinter窗口上的鼠标点击，并将其映射到目标移动位置。同时也可以使用键盘操作。进阶功能：使用线程实现玩家与NPC的实时同步移动。

恭喜！如果你已经走到这一步，说明你已经成功创建了一个完整的MVC风格的应用程序。太好了！

4. 提交要求

提交运行你的解决方案所需的所有Python源代码文件，我们假设你仅使用了Python标准库中的库。你不得使用外部库，否则我们将无法正确评估你的解决方案。